



What is Amazon Simple Queue Service?

Amazon Simple Queue Service

Amazon Simple Queue Service (Amazon SQS) offers a secure, durable, and available hosted queue that lets you integrate and decouple distributed software systems and components. Amazon SQS offers common constructs such as dead-letter queues and cost allocation tags. It provides a generic web services API that you can access using any programming language that the AWS SDK supports.

Amazon SQS supports both standard and FIFO queues.

Benefits of using Amazon SQS

- **Security** – You control who can send messages to and receive messages from an Amazon SQS queue. Server-side encryption (SSE) lets you transmit sensitive data by protecting the contents of messages in queues using keys managed in AWS Key Management Service (AWS KMS).
- **Durability** – For the safety of your messages, Amazon SQS stores them on multiple servers. Standard queues support at-least-once message delivery, and FIFO queues support exactly-once message processing.
- **Availability** – Amazon SQS uses redundant infrastructure to provide highly-concurrent access to messages and high availability for producing and consuming messages.
- **Scalability** – Amazon SQS can process each buffered request independently, scaling transparently to handle any load increases or spikes without any provisioning instructions.
- **Reliability** – Amazon SQS locks your messages during processing, so that multiple producers can send and multiple consumers can receive messages at the same time.
- **Customization** – Your queues don't have to be exactly alike—for example, you can set a default delay on a queue. You can store the contents of messages larger than 256 KB using Amazon Simple Storage Service (Amazon S3) or Amazon DynamoDB, with Amazon SQS holding a pointer to the Amazon S3 object, or you can split a large message into smaller messages.

Differences between Amazon SQS, Amazon MQ, and Amazon SNS

Amazon SQS and Amazon SNS are queue and topic services that are highly scalable, simple to use, and don't require you to set up message brokers. We recommend these services for new applications that can benefit from nearly unlimited scalability and simple APIs.

Amazon MQ is a managed message broker service that provides compatibility with many popular message brokers. We recommend Amazon MQ for migrating applications from existing message brokers that rely on compatibility with APIs such as JMS or protocols such as AMQP, MQTT, OpenWire, and STOMP.

Queue types

The following table describes the capabilities of standard queues and FIFO queues.

Standard queue	FIFO queue
Unlimited Throughput – Standard queues support a nearly unlimited number of API calls per second, per API action (SendMessage, ReceiveMessage, or DeleteMessage). At-Least-Once Delivery – A message is delivered at least once, but occasionally more than one copy of a message is delivered. Best-Effort Ordering – Occasionally, messages are delivered in an order different from which they were sent. 	High Throughput – If you use batching, FIFO queues support up to 3,000 transactions per second, per API method (SendMessageBatch, ReceiveMessage, or DeleteMessageBatch). The 3000 transactions represent 300 API calls, each with a batch of 10 messages. To request a quota increase, submit a support request. Without batching, FIFO queues support up to 300 API calls per second, per API method (SendMessage, ReceiveMessage, or DeleteMessage). Exactly-Once Processing – A message is delivered once and remains available until a consumer processes and deletes it. Duplicates aren't introduced into the queue. First-In-First-Out Delivery – The order in which messages are sent and received is strictly preserved. 
Send data between applications when the throughput is important, for example: - Decouple live user requests from intensive background work: let users upload media while resizing or encoding it. - Allocate tasks to multiple worker nodes; process a high number of	Send data between applications when the order of events is important, for example: - Make sure that user-entered commands are run in the right order. - Display the correct product price by sending price modifications in the right order. - Prevent a student from enrolling in a course before registering for an account.

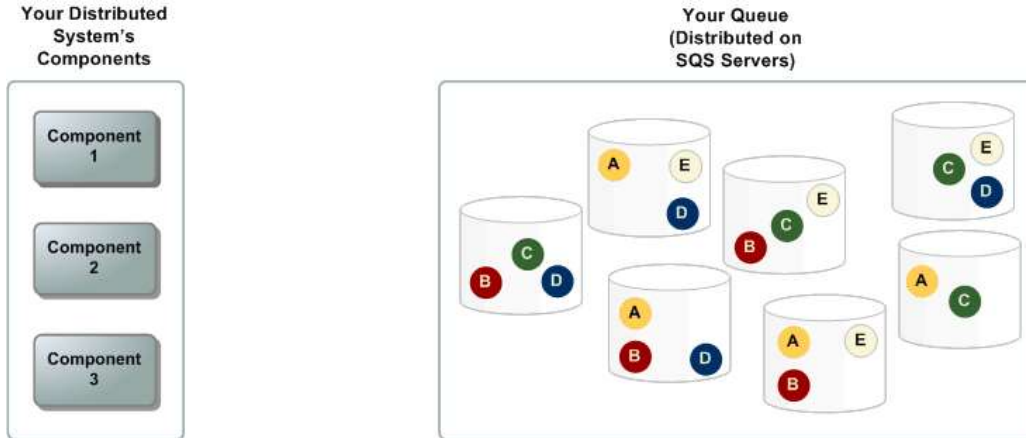


Basic Amazon SQS architecture

Distributed queues

There are three main parts in a distributed messaging system: the components of your distributed system, your queue (distributed on Amazon SQS servers), and the messages in the queue.

In the following scenario, your system has several *producers* (components that send messages to the queue) and *consumers* (components that receive messages from the queue). The queue (which holds messages A through E) redundantly stores the messages across multiple Amazon SQS servers.

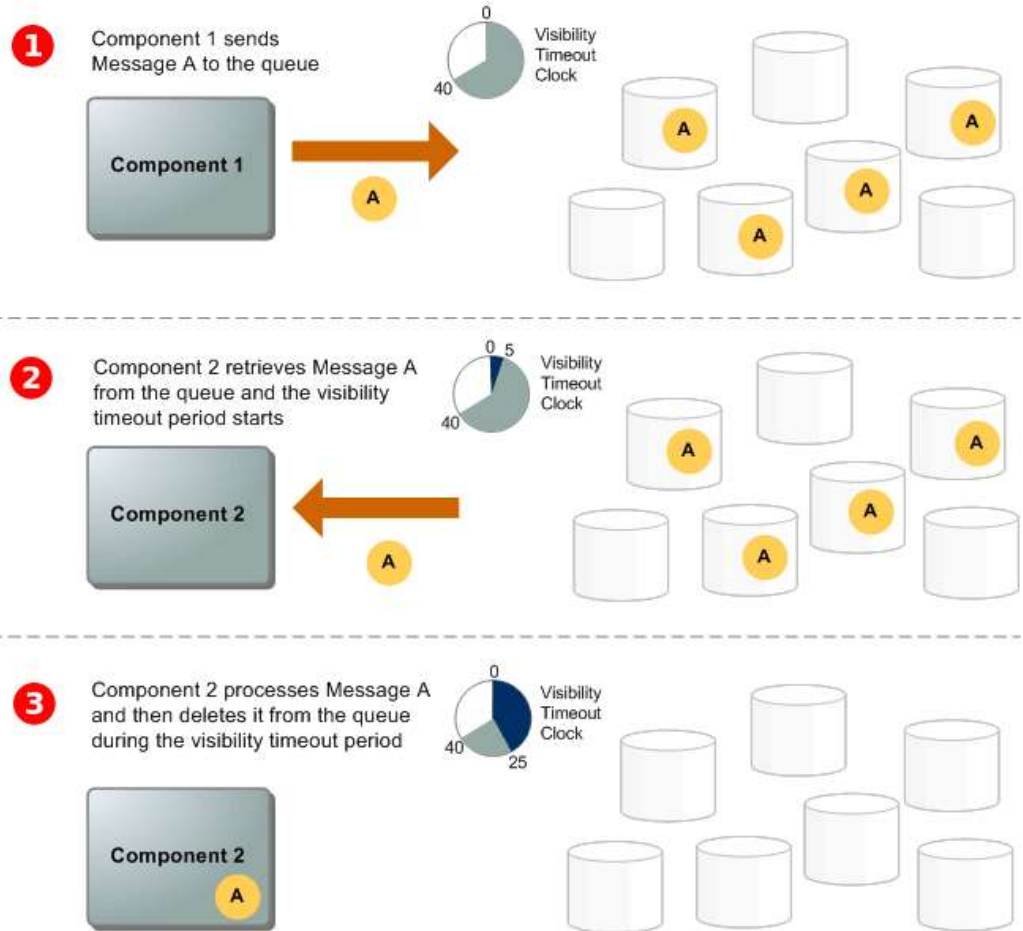


Message lifecycle

- 1 A producer (component 1) sends message A to a queue, and the message is distributed across the Amazon SQS servers redundantly.

- 2 When a consumer (component 2) is ready to process messages, it consumes messages from the queue, and message A is returned. While message A is being processed, it remains in the queue and isn't returned to subsequent receive requests for the duration of the **visibility timeout**.

- 3 The consumer (component 2) deletes message A from the queue to prevent the message from being received and processed again when the visibility timeout expires.





Amazon SQS Standard queues

Amazon SQS Standard queues

Amazon SQS offers *standard* as the default queue type. Standard queues support a nearly unlimited number of API calls per second, per API action (`SendMessage`, `ReceiveMessage`, or `DeleteMessage`). Standard queues support at-least-once message delivery. However, occasionally (because of the highly distributed architecture that allows nearly unlimited throughput), more than one copy of a message might be delivered out of order. Standard queues provide best-effort ordering which ensures that messages are generally delivered in the same order as they're sent.

You can use standard message queues in many scenarios, as long as your application can process messages that arrive more than once and out of order, for example:

- **Decouple live user requests from intensive background work** – Let users upload media while resizing or encoding it.
- **Allocate tasks to multiple worker nodes** – Process a high number of credit card validation requests.
- **Batch messages for future processing** – Schedule multiple entries to be added to a database.

Message ordering

A standard queue makes a best effort to preserve the order of messages, but more than one copy of a message might be delivered out of order. If your system requires that order be preserved, I recommend using a FIFO (First-In-First-Out) queue or adding sequencing information in each message so you can reorder the messages when they're received.

At-least-once delivery

Amazon SQS stores copies of your messages on multiple servers for redundancy and high availability. On rare occasions, one of the servers that stores a copy of a message might be unavailable when you receive or delete a message.

If this occurs, the copy of the message isn't deleted on that unavailable server, and you might get that message copy again when you receive messages. Design your applications to be idempotent (they should not be affected adversely when processing the same message more than once).



Getting started with Amazon SQS

Step 1: Create a queue

The first and most common Amazon SQS task is creating queues. This procedure shows how to create and configure a FIFO queue.

1. Open the Amazon SQS console at <https://console.aws.amazon.com/sqs/>.
2. Choose **Create queue**.
3. On the **Create queue** page, specify the correct region.
4. The **Standard** queue type is selected by default. Choose **FIFO**.

You can't change the queue type after you create a queue.

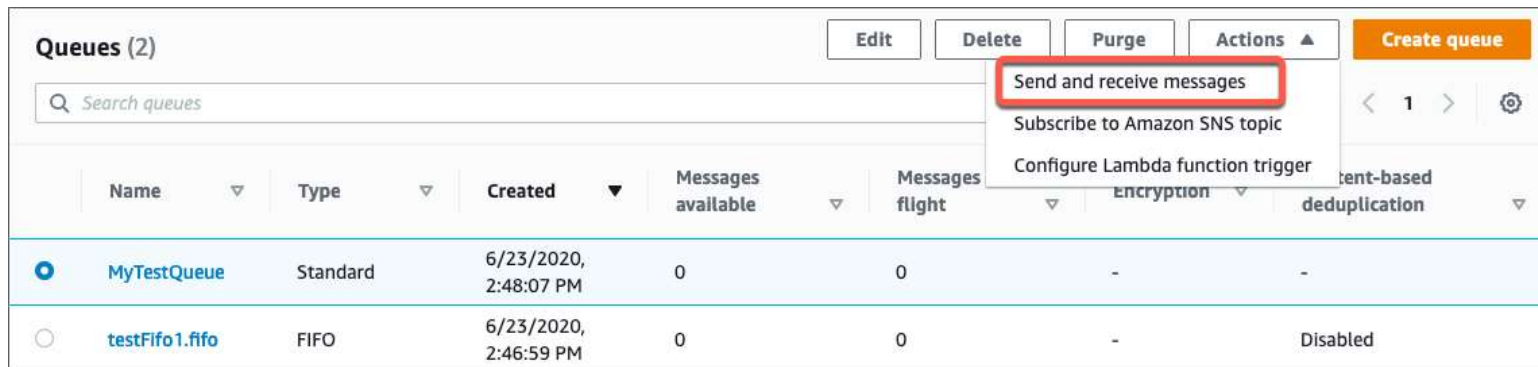
5. Enter a **Name** for your queue. The name of a FIFO queue must end with the `.fifo` suffix.
6. To create your queue with the default parameters, scroll to the bottom and choose **Create Queue**. Amazon SQS creates the queue and displays the queue's **Details** page.

Amazon SQS propagates information about the new queue across the system. Because Amazon SQS is a distributed system, you may experience a slight delay before the queue is displayed on the **Queues** page.

Step 2: Send a message

After you create your queue, you can send a message to it.

1. From the left navigation pane, choose **Queues**. From the queue list, select the queue that you created.
2. From **Actions**, choose **Send and receive messages**. The console displays the **Send and receive messages** page.



Queues (2)

Search queues

Buttons: Edit, Delete, Purge, Actions, Create queue

Dropdown menu options: Send and receive messages, Subscribe to Amazon SNS topic, Configure Lambda function trigger

	Name	Type	Created	Messages available	Messages in flight	Encryption	Content-based deduplication
☒	MyTestQueue	Standard	6/23/2020, 2:48:07 PM	0	0	-	-
☐	testFifo1.fifo	FIFO	6/23/2020, 2:46:59 PM	0	0	-	Disabled

Step 2: Send a message

3. Enter text in the **Message body**
4. Enter a **Message group id** for the queue. For more information, see FIFO delivery logic.
5. (Optional) Enter a **Message deduplication id**. If you enable content-based deduplication, the message deduplication ID is not required. For more information, see FIFO delivery logic.
6. Choose **Send message**.

Your message is sent and the console displays a success message. Choose **View details** to display information about the sent message.

Send message

Clear content

Send message

✔ Your message has been sent and is ready to be received.

View details

×

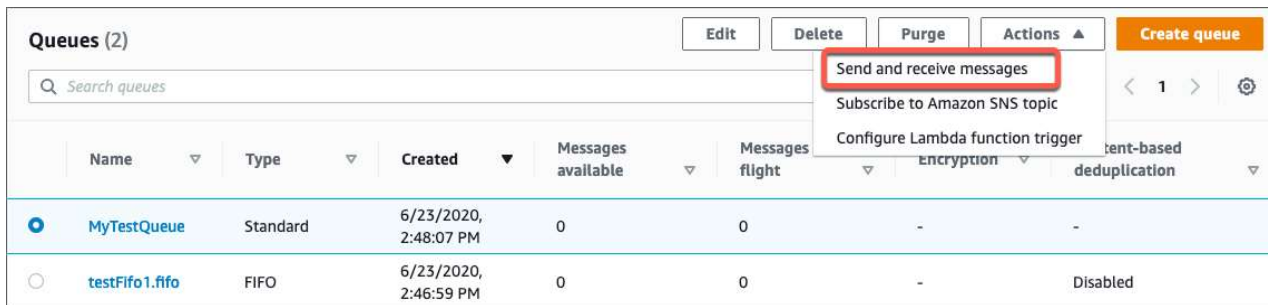
Message body

Enter the message to send to the queue.

Step 3: Receive and delete your message

After you send a message to a queue, you can retrieve the message from the queue. When you request messages from a queue, you can't specify which message to retrieve. Instead, you specify the maximum number of messages (up to 10) that you want to retrieve.

1. From the **Queues** page, select a queue.
2. From **Queue Actions**, select **Send and receive messages**. The console displays the **Send and receive messages** page.



The screenshot shows the AWS IAM console 'Queues' page. At the top, there are buttons for 'Edit', 'Delete', 'Purge', 'Actions', and 'Create queue'. Below these is a search bar labeled 'Search queues'. The main part of the page is a table with columns: Name, Type, Created, Messages available, Messages in flight, Encryption, and Content-based deduplication. Two queues are listed: 'MyTestQueue' (Standard type, created 6/23/2020, 2:48:07 PM, 0 messages available, 0 in flight, no encryption, no deduplication) and 'testFifo1.fifo' (FIFO type, created 6/23/2020, 2:46:59 PM, 0 messages available, 0 in flight, no encryption, deduplication is 'Disabled'). The 'Actions' button is expanded, showing a dropdown menu with three options: 'Send and receive messages' (highlighted with a red box), 'Subscribe to Amazon SNS topic', and 'Configure Lambda function trigger'.

Name	Type	Created	Messages available	Messages in flight	Encryption	Content-based deduplication
MyTestQueue	Standard	6/23/2020, 2:48:07 PM	0	0	-	-
testFifo1.fifo	FIFO	6/23/2020, 2:46:59 PM	0	0	-	Disabled

Step 3: Receive and delete your message

3. Choose **Poll for messages**.

Amazon SQS begins to poll servers to find messages in the queue. The progress bar on the right side of the **Receive messages** section displays the polling duration.

The **Messages** section displays a list of the received messages. For each message, the list displays the message ID, sent date, size, and receive count.

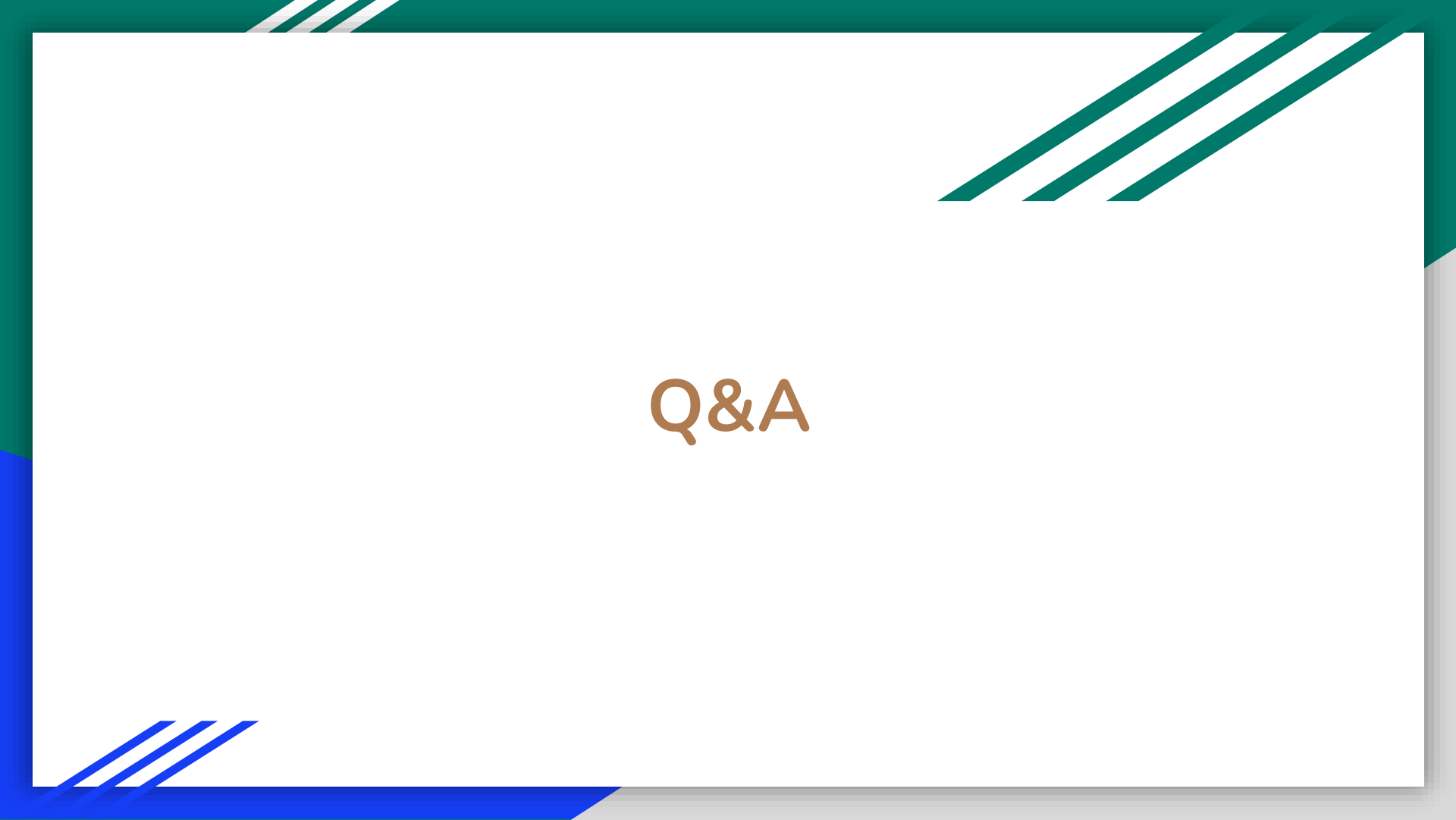
4. To delete messages, select the messages that you want to delete and then choose **Delete**.
5. In the **Delete Messages** dialog box, choose **Delete**.

Step 4: Delete your queue

1. From the queue list, select the queue that you have created.
2. From the **Queues** page, select the queue to delete.
3. Choose **Delete queue**.

The console displays the **Delete queue** dialog box.

4. In the **Delete queue** dialog box, confirm the deletion by entering **delete**.
5. Choose **Delete**.



Q&A